Software Testing (CSA37)

ASSIGNMENT 1

CO1: Testing as an engineering activity, role of process in software quality, testing as a process, basic definitions, software testing principles, tester's role, origins of defects, defect classes, defect repository, defect examples.

Assignment Title: Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application (Total Marks - 50)

Duration:1week

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident

I1	**Two patients receive the same token number for the same doctor when they book at the same second.**
I2	**Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.**
I3	**Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood.**
I4	**Application crashes when a patient name has more than 40 characters.**
I5	**Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.**
I6	**Prescription PDF shows the previous patient's details when two doctors save at the same time.**
I7	**The user manual tells patients to cancel through "My Visits", but this menu does not exist.**
I8	**A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.**
I9	**SMS reminders are not sent when the hospital's SMS provider changes its API key.**
I10	**The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.**

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect (₹)	Remarks
Requirements	30	2	500	No formal review
Design	22	3	1,500	No formal review
Coding	38	15	3,000	Developer self-check only
Testing	5	20	8,000	Demo-based testing
Documentation	5	0	–	–
Production (after release)	–	60	40,000	Reported by patients

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.

Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate Testing as an Engineering Activity 5 Marks

CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity.

Did the MediQueue team engineer quality into the product, or did they only hope for it?

Discuss:

- **The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)**
- **Which of these characteristics were missing in the MediQueue project, with evidence from the case**
- **How the absence of each characteristic contributed to the incidents reported**
- **What an engineered testing approach would have looked like for this project**

You are expected to judge the project against engineering practice, not merely define the term.

Evidence Expected

A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

TASK 2 – Evaluate the Role of Process in Software Quality 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem.

Is quality mainly a result of good people, or of a good process?

Discuss:

- **The role of a defined process in achieving software quality**
- **Whether better developers alone would have prevented the incidents**
- **How a process maturity view (for example, TMM levels) would place the current team**

Your position must be defended with evidence from the case.

Evidence Expected

A justified position on the role of process in quality, supported by the case data.

TASK 3 – Evaluate Testing as a Process 6 Marks

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release.

What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Discuss:

- **Testing as a process with planning, design, execution, evaluation and reporting activities**
- **The relationship between verification and validation activities and each development phase**
- **The phase in which each type of MediQueue defect should have been detected**
- **Using the defect data table, the cost the hospital paid because defects were found late**
- **The likely saving if requirements and design reviews had been conducted**
- **The limits of your estimate and the assumptions it depends on**

You must show your calculations and interpret them. Stating that "early testing is cheaper" will not receive full credit.

Evidence Expected

A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

TASK 4 – Apply and Critique Basic Definitions 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

The team uses the words "bug", "error" and "failure" interchangeably in its reports.

Does careless terminology actually harm quality, or is it only a matter of words?

Discuss:

- **The meaning of error, fault (defect), failure, test case, test oracle and test bed in the MediQueue context**
- **For incidents I3, I5 and I6, the error, the fault and the failure**
- **Why the confusion in terminology made incident I8 possible**
- **Whether the team needs a common glossary, and what it must contain**

Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.

Evidence Expected

Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

TASK 5 – Evaluate the Application of Testing Principles 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The developers claim that "we tested everything and found no errors, so the software was correct".

Which testing principles prove this claim wrong?

Discuss:

- **At least four software testing principles that apply to the MediQueue situation**
- **For each principle, the incident or decision in the case that violates or confirms it**
- **Which principle, if followed, would have had the largest effect on quality, and why**

Do not simply list principles. You are expected to relate each principle to evidence.

Evidence Expected

A justified evaluation of the developers' claim using testing principles linked to evidence from the case.

TASK 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester.

Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Discuss:

- **The role and responsibilities of a tester across the life cycle**
- **The advantages and limitations of developers testing their own code**
- **Which incidents a tester would most likely have caught, and at which stage**
- **The qualities a tester for a healthcare application must have**

Your answer must take a position and defend it.

Evidence Expected

A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

TASK 7 – Trace the Origins of Defects 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated.

Where in the life cycle are MediQueue's defects really coming from?

Discuss:

- **The origin of each incident: requirements, design, code, testing, documentation or environment**

- **The evidence in the case that supports each origin you assign**
- **Incidents that may have more than one origin, and how you decided**
- **What the overall pattern of origins tells the hospital about where to invest effort**

Where an incident could belong to more than one origin, argue for your choice.

Evidence Expected

A justified origin for each incident and an interpretation of the overall pattern of defect origins.

TASK 8 – Evaluate Defect Classes and Severity 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects.

Which classes of defects are most harmful to patients, and do the team's priorities reflect that?

Discuss:

- **A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)**
- **The class, severity and priority of each incident, with reasons**
- **Why severity and priority may differ for the same defect (for example, I7 and I1)**
- **Which defect class the team should target first, and why**

Simply placing defects in a table is not sufficient. You are expected to justify each classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

TASK 9 – Design and Appraise a Defect Repository 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet.

What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Discuss:

- **The fields a defect repository for MediQueue must contain, and why each is needed**
- **How the repository supports defect analysis, process improvement and prevention**
- **The weaknesses of the current e-mail and spreadsheet approach**

You are expected to justify your repository design, not just list fields.

Evidence Expected

A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes.

For each selected defect, what single change to the process would have prevented it?

Discuss:

- **The root cause of each selected defect**
- **A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)**
- **The cost and effort of each strategy compared with its benefit**
- **Which strategy gives the best value for the hospital**

Your recommendation must be supported by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified, cost-aware prevention strategies.

TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.

Discuss:

- **The process changes you recommend and their order of priority**
- **The role of testers, reviews and the defect repository in your plan**
- **The expected effect on the number and cost of defects**
- **Trade-offs, risks and limitations of your plan**
- **Whether the plan remains practical for a small team of five developers**

Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.

Evidence Expected

A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Quality Process 8 Marks

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.

At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.

Patient health records will now be shared between hospitals.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed from the original situation**
- **Which assumptions in your quality improvement plan are affected**
- **Why frequent releases change the role of testing and the tester**
- **Why sharing health records raises the severity of some defect classes**

Evidence Expected

A clear analysis of how the changed conditions affect the original quality improvement plan.

B. Evaluation of the Original Plan – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended plan remains appropriate.

Discuss:

- **Whether the planned reviews and test activities fit into a two-week cycle**

- **Which new defect origins are introduced by the external health ID system**
- **Whether your defect repository is still sufficient**

Evidence Expected

A technically supported evaluation of the suitability of the original plan under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- **Describe the revised process and the tester's revised role**
- **Explain which testing principles now become more important**
- **Explain how defects from the external system will be classified and tracked**

If you retain the original plan, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate response to the changed requirement with clear quality-engineering reasoning.

D. Evidence-Based Defence – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Use:

- **The defect data and cost figures**
- **Defect origins and classes**
- **Release frequency**

- **Patient-safety and data-privacy risk**
- **Team size and capacity**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Evaluate Testing as an Engineering Activity	**5**
Task 2 – Evaluate the Role of Process in Software Quality	**2**
Task 3 – Evaluate Testing as a Process	**6**
Task 4 – Apply and Critique Basic Definitions	**4**
Task 5 – Evaluate the Application of Testing Principles	**3**
Task 6 – Evaluate the Tester's Role	**3**
Task 7 – Trace the Origins of Defects	**4**

Task 8 – Evaluate Defect Classes and Severity	**4**
Task 9 – Design and Appraise a Defect Repository	**3**
Task 10 – Evaluate Defect Examples and Prevention Strategies	**4**
Task 11 – Recommend and Defend a Quality Improvement Plan	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

ANSWERS

TASK 1 – Evaluate Testing as an Engineering Activity (5 Marks)

Ans:

Position: MediQueue's testing was not an engineering activity. Quality was hoped for, not engineered. The developers tested their own code just before release, and the sign-off rested on a demo with "no errors". A demo only shows that the paths someone chose to show work. It says nothing about untried paths.

Table 1: MediQueue testing against the characteristics of engineering practice

Characteristic	**What engineering practice expects**	**Evidence in the case**	**Incidents it contributed to**
Planning	Test goals, levels, risks, schedule and exit criteria written down before testing	No test plan. Testing happened "just before release", after 8 months of building	I1, I6, I9, I10
Measurement	Coverage, defects per phase and cost are tracked and drive the release decision	Release criterion was "demo showed no errors". Only 40 of 100 defects were found before release, and nothing flagged that as unacceptable	All ten
Standards / defined process	Reviews, standard test-case format, traceability to requirements	Requirements and design had "no formal review". Coding had only "developer self-check"	I2, I3, I7
Repeatability	Documented test cases, data and expected results that anyone can re-run	Demo-based testing cannot be repeated. The one recorded test (refund) used a wrong expected value	I4, I5, I8

Independence and roles	A person whose job is to find defects	No tester. Developers' assumptions were used to write and to check the code	I5, I6
Tools / infrastructure	Test environment, test data, defect tracking	Complaints were kept in e-mails and a spreadsheet. No test bed for the SMS provider or load	I9, I10

How each missing feature led to incidents

- **No planning:** nobody asked what happens on a slow network (I10), with two users in the same second (I1, I6), or when the SMS key changes (I9).
- **No measurement:** the team could not see its risk. Only 5 of the 52 requirement and design defects were found early, and the 20 defects found in testing were about a quarter of the 75 present at that stage.
- **No standards or reviews:** nobody checked the Sunday rule or the GST rule with the people who own them (I2, I3). The wrong menu name in the manual (I7) is the same gap in document review.
- **No repeatability:** a 41-character name (I4) and a patient aged exactly 60 (I5) were only checked by chance. When a test was written (refund), nobody checked its expected value (I8).
- **No independent tester:** the person who writes the code shares its assumptions. A developer who believes "two doctors never save together" will never test it (I6).

What an engineered approach would have looked like

- A test plan per release covering levels (unit, integration, system, acceptance), risks and schedule.
- Exit criteria, for example: all high-risk tests executed, no open critical or high defect, at least 95% of planned tests passed.
- Requirement review with billing and OPD staff, and design review of concurrency and shared data, before coding.

- Test cases from equivalence classes and boundary values, each with an expected result traced to a requirement.
- System tests for concurrency, performance on hospital Wi-Fi and the SMS interface, run in a proper test bed.
- A defect repository giving defects per phase, cost and escapes for every release.

Conclusion: testing was a single event, not a process. Each missing feature links to at least one incident, so the incidents were foreseeable results of how the work was organised, not bad luck.

TASK 2 – Evaluate the Role of Process in Software Quality (2 Marks)

Ans:

Position: quality comes mainly from process. Better developers help, but most MediQueue defects arose before coding or outside the code, where developer skill cannot reach.

How a defined process helps: it adds checks at every phase (reviews, planned tests, independent testing, defect tracking). It catches mistakes that skilled people also make, and it makes results repeatable instead of dependent on individuals.

Table 2: Would a better developer alone have prevented the incident?

Incident	Prevented by developer skill alone?	Reason
I4, I5	Probably	Input-length and age boundaries are coding skill, but a boundary test would catch them even with an average developer
I1, I6	Partly	An experienced developer may foresee concurrency, but a design review and a concurrency test are still needed to be sure
I10	Partly	Needs a performance target (requirement) and a load test

I2, I3	No	The Sunday rule was never written and the GST rule was misread. Skill cannot supply a rule that was never given
I7	No	Documentation defect, needs a document review
I8	No	Testing defect, needs test-case review by a second person
I9	No	External change, needs interface monitoring and configuration control

Skill alone could prevent at most 4 of the 10 incidents. The data agree: only 38 of 100 defects were introduced in coding, while 52 came from requirements and design. If better developers halved coding defects, 19 would disappear and 81 would remain.

Maturity position (TMM): the team is at **Level 1 (Initial)**. Testing is not separated from debugging, its only aim is to show the software works (the demo), and there is no plan or trained tester. The target should be **Level 2 (Phase Definition)**: defined testing goals, test planning and basic techniques. Level 3 (Integration) comes next, with a test group and a test life cycle tied to development.

TASK 3 – Evaluate Testing as a Process (6 Marks)

Ans:

Position: if testing ran through the whole life cycle, most defects would have been found at ₹500–₹3,000 each instead of ₹40,000 each in production.

1. Testing as a process

Table 3: Test process activities for MediQueue

Activity	**What it involves in MediQueue**	**Output**
Planning	Test goals, levels, risks (money, concurrency, privacy), schedule, exit criteria	Test plan

Design	Cases from requirements: age 59/60/61, name length 40/41/42, Sunday booking, fee shown = fee charged, two bookings in the same second	Test cases with expected results
Set-up	Test doctors/patients, SMS sandbox, Wi-Fi conditions	Test bed
Execution	Run planned tests at unit, integration, system level; re-run after every fix	Test log
Evaluation	Compare actual with expected (oracle), check exit criteria	Pass/fail verdicts, defect reports
Reporting and closure	Test summary, lessons learnt, repository entries	Summary report

2. Verification and validation by phase

Table 4: V&V activities

Phase	**Verification (building it right?)**	**Validation (building the right thing?)**	**Defects targeted**
Requirements	Checklist review, testability check	Acceptance tests planned with hospital staff	I2, I3, I10
Design	Design inspection (concurrency, shared data, interfaces)	System tests designed	I1, I6, I9
Coding	Code review, standards, boundary unit tests	–	I4, I5
Integration/system	Interface and regression tests, test-case review	Performance, concurrency and acceptance tests	I1, I3, I6, I9, I10
Documentation	Check manual against the working system	A real user follows the manual	I7

Operation	Interface monitoring, regression after changes	Patient feedback	I9

3. Phase in which each defect should have been detected

Table 5

Incident	Should have been detected in	How
I1	Design	Design inspection (atomic token allocation); concurrent-booking test
I2	Requirements	Review with OPD staff
I3	Requirements	Review with billing; test comparing displayed and charged fee
I4	Coding	Unit test with name lengths 40/41
I5	Coding	Boundary test 59/60/61; code review
I6	Design	Inspection of shared data; concurrent-save test
I7	Documentation	Review of manual against the application
I8	Testing	Second-person review of the test case
I9	Design / integration	Interface test, configuration control, monitoring
I10	Requirements / system test	Performance target; load test

4. Cost of late detection

Table 6: Cost of the last release's 100 defects (from the case data)

Phase detected	Defects	Cost each (₹)	Total (₹)	Share of cost
Requirements	2	500	1,000	0.04%

Design	3	1,500	4,500	0.17%
Coding	15	3,000	45,000	1.72%
Testing	20	8,000	1,60,000	6.13%
Production	60	40,000	24,00,000	91.93%
Total	**100**		**26,10,500**	100%

(Figure 1: draw a bar chart of the "Total (₹)" column by phase. The production bar dwarfs the rest.)

Calculation: 2×500 + 3×1,500 + 15×3,000 + 20×8,000 + 60×40,000 = 1,000 + 4,500 + 45,000 + 1,60,000 + 24,00,000 = **₹26,10,500**.

Ideal cost (each of the 95 defects that have a unit cost fixed in the phase that introduced it): 30×500 + 22×1,500 + 38×3,000 + 5×8,000 = 15,000 + 33,000 + 1,14,000 + 40,000 = **₹2,02,000**. Documentation defects have no stated cost, so they are excluded.

Interpretation: patients found 60% of the defects, and these made up 91.9% of the cost. A production defect costs 80 times a requirements defect (40,000 ÷ 500) and 5 times a testing defect. Late detection cost about ₹24 lakh more than ideal (26,10,500 − 2,02,000), roughly 13 times the ideal figure.

Table 7: Where defects were caught (defects missed in one phase pass to the next)

Phase	**Introduced**	**Present**	**Detected**	**Detection efficiency**	**Passed on**
Requirements	30	30	2	6.7%	28
Design	22	50	3	6.0%	47
Coding	38	85	15	17.6%	70
Testing	5	75	20	26.7%	55
Documentation	5	60	0	0%	60
Production	–	60	60	100%	0

Requirements and design received 52 defects and caught only 5 (9.6%). Only 40% of defects were found before release, so the "no errors" demo hid 60 defects.

5. Probable saving from requirements and design reviews

Assumptions: reviews remove 50% of the defects present in the requirements and design phases. Coding and testing keep their current rates (17.6% and 26.7%). The two reviews cost ₹1,00,000 (about 125 person-hours at ₹800). Documentation defects are treated as reaching production.

Phase	**Present**	**Detected**	**Cost (₹)**	**Passed on**
Requirements	30	15.0	7,500	15.0
Design	15 + 22 = 37	18.5	27,750	18.5
Coding	18.5 + 38 = 56.5	9.97	29,912	46.53
Testing	46.53 + 5 = 51.53	13.74	1,09,929	37.79
Production	37.79 + 5 = 42.79	42.79	17,11,529	–
Total			**≈ 18,86,620**	

Gross saving = 26,10,500 − 18,86,620 ≈ **₹7.24 lakh**. **Net saving** = 7,24,000 − 1,00,000 ≈ **₹6.24 lakh (about 24%)**. Defects reaching patients fall from 60 to about 43. Reviews pay for themselves because ₹1 lakh spent avoids many ₹40,000 production fixes.

6. Limitations

- The case does not give the origin of each detected defect, so I assumed the pass-on flow in Table 7.
- The 50% review efficiency is an assumption. The team has no review data.
- Unit costs cover rework only. Patient harm, lost trust, legal exposure and hospital staff time are excluded, so real costs are higher.
- The ₹1,00,000 review cost is my estimate.

- The data come from one release of one team. Defects never found are not counted.

TASK 4 – Apply and Critique Basic Definitions (4 Marks)

Ans:

Position: loose terminology does harm quality. If "bug", "error" and "failure" mean the same thing, the team cannot tell whether the problem is in a person's thinking, in the product or in what the user sees, so it cannot trace or prevent the cause.

Table 8: Definitions applied to MediQueue

Term	Meaning	MediQueue example
Error	A mistake made by a person	A developer assumes two doctors never save at the same moment
Fault (defect, bug)	The flaw in code or document caused by an error	Prescription data is held in a variable shared by all doctors
Failure	Wrong behaviour seen when faulty code runs	The PDF shows the previous patient's details
Test case	Inputs, conditions and expected result for one check	Book a Sunday appointment → system rejects it with "OPD closed"
Test oracle	The source that decides the expected result	The approved fee table says fee = ₹500 + GST, which decides whether ₹550 is correct
Test bed	Environment where tests run	Copy of the app with test doctors, an SMS sandbox and Wi-Fi-like network conditions

Table 9: Error, fault, failure for I3, I5, I6

Incident	Error (human mistake)	Fault (in the product)	Failure (user sees)

I3	GST rule misunderstood and applied differently in two places	Booking page uses the base fee while the payment module adds tax (two separate fee calculations)	Page shows ₹500, patient pays ₹550
I5	"Turns 60" read as "over 60", or age computed on the wrong date	Condition like age > 60, or age taken on booking date instead of appointment date	Patient turning 60 on the day gets no priority
I6	Assumed two doctors never save at once	Patient details stored in shared data that the second save overwrites	PDF shows the previous patient

How confusion allowed I8: the tester made an error that put a fault into the test case (a wrong expected value), which produced a false "Pass". A team that calls everything a "bug" never separates a fault in the product from a fault in the test. No oracle was defined, so the expected value came from memory instead of the requirement, and nobody asked where it came from. The refund defect stayed hidden.

Why it harms quality:

1. Root-cause analysis breaks down, because an error, a fault and a failure need different remedies (training, review checklist, code fix).
2. Counts are distorted: several patient complaints (failures) may come from one fault.
3. Management cannot judge seriousness from the word "bug".
4. Fix verification is weak, because nobody records which test case and oracle prove the fault is gone.

Glossary needed? Yes, one page, agreed by developers, testers and hospital staff and kept with the defect repository.

Table 10: Glossary contents

Group	**Terms**	**Why**
Defect terms	error, fault, failure, incident	Separate cause, flaw and symptom

Test terms	test case, expected result, oracle, test bed, test log, regression test	Avoid wrong expected values (I8)
Ratings	severity and priority levels with MediQueue examples	End arguments about what to fix first
Defect states	new, assigned, fixed, verified, closed, reopened	Track to closure
Product terms	appointment day, senior citizen, fee, token, release	Remove ambiguity such as “senior at 60” (I5)

TASK 5 – Evaluate the Application of Testing Principles (3 Marks)

Ans:

Position: the claim “we tested everything and found no errors, so the software was correct” fails twice. It treats no failures in the tests that were run as proof of correctness, and it assumes every possible test was run.

Table 11: Principles applied to MediQueue

Principle	**Violated / confirmed**	**Evidence from the case**
Testing shows the presence of defects, never their absence	Violated	A demo with no errors was taken as proof of correctness, yet 60 defects were found by patients afterwards
Exhaustive testing is impossible, so tests must be chosen by risk	Violated	No risk-based selection. Money, concurrency and privacy paths were not targeted (I1, I3, I6)

Test early, throughout the life cycle	Violated	No review of requirements or design. 52 of 100 defects started there and 5 were caught early. Late detection cost ₹26.1 lakh
Testing must be planned	Violated	No plan, so no time for load, concurrency or SMS tests (I9, I10)
Test valid and invalid inputs, with expected results stated	Violated	41-character name (I4), Sunday booking (I2), and a wrong expected value in the refund test (I8)
Testing should be independent	Violated	Developers tested their own code, so assumptions like "single user" and "age > 60" passed unchallenged (I5, I6)
Defects cluster	Confirmed	I1 and I6 share one cause, shared data used at the same moment. Finding one should have triggered a search for the other

Greatest impact: *test early, throughout the life cycle.* Half of the incidents (I1, I2, I3, I6, I10) originate in requirements and design, and those phases caught only 9.6% of their defects. Cost per defect climbs from ₹500 to ₹40,000. Planning and independence help, but both work best if they start at requirements. Writing test cases from the requirements would have exposed the missing Sunday rule and the fee mismatch before any code existed.

TASK 6 – Evaluate the Tester's Role (3 Marks)

Ans:

Position: an independent tester would have changed the outcome substantially but not completely. Most incidents were findable by a tester, while I1, I6 and I9 also need design review, specialised tests and monitoring.

Role across the life cycle

- **Requirements:** review for missing, unclear or untestable statements ("What are OPD working days? Is the fee tax-inclusive? How fast must the schedule load?").
- **Design/planning:** prepare the test plan, risks, exit criteria, test cases and test bed.
- **Coding/integration:** support unit testing, run integration tests, maintain the regression suite.
- **System/acceptance:** run functional, load, concurrency and security tests, and support user acceptance.
- **Throughout:** report defects with steps to reproduce, verify fixes, keep the repository and give evidence for the release decision.

Developers testing their own code

- *Advantages:* they know the code, find simple faults quickly and fix them cheaply. Developer unit testing should continue.
- *Limitations:* they test what they think the code does, so a wrong assumption is carried into the test. They want to show their work is good, testing gets squeezed to the end, and nobody checks their reading of the requirement. I5 and I6 both come from the developers' own assumptions.

Table 12: Incidents a tester would probably have caught

Incident	Caught?	Stage and how
I2	Yes	Requirement review: asks which days the OPD works
I3	Yes	Requirement review and a test comparing displayed and charged fee
I4	Yes	Boundary-length name test (40, 41)
I5	Yes	Boundary test 59/60/61 on the appointment day
I7	Yes	Documentation review by following the manual on the real app
I8	Yes	Second-person review of the test case, oracle traced to the requirement

I10	Yes	Load test on hospital Wi-Fi, once a performance target exists
I1	Maybe	Only if a concurrency test is designed. Design inspection is the stronger safeguard
I6	Maybe	Needs a concurrent-save test and a design review of shared data
I9	Partly	Interface test in an SMS sandbox. The later key change needs monitoring

Effect on the data: suppose a tester raises testing-phase detection from 26.7% to 60%, with no reviews. Testing then finds 45 of the 75 defects present. Production defects fall from 60 to 35. Cost = 1,000 + 4,500 + 45,000 + 45×8,000 + 35×40,000 = ₹19,10,500, a saving of **₹7,00,000**. That is a big gain, but 35 defects still reach patients, so the tester must work together with reviews.

Qualities for healthcare testing

- **Domain knowledge:** outpatient workflow, fees and tax, senior-citizen rules, prescriptions.
- **Attention to detail:** notices a ₹50 difference, a wrong menu name, a wrong expected value.
- **Sensitivity to safety and privacy:** knows that showing one patient's data to another (I6) is a serious breach.
- **Technical skill:** test design techniques, concurrency and load testing, defect tracking, basic automation.
- **Communication and ethics:** reports without blame, keeps health data confidential, and is willing to say "not ready for release".

TASK 7 – Trace the Origins of Defects (4 Marks)

Ans:

Table 13: Origin of each incident

Incident	**Main origin**	**Evidence**	**Possible second origin, and why I chose the main one**

I1	Design	Same token only when two bookings land in the same second, so token allocation was not designed as one atomic step	Coding. The weakness is in how allocation is structured, not a typing slip
I2	Requirements	Case states the rule was never written	None
I3	Requirements	GST rule “misunderstood”, so the business rule was wrong or unclear	Coding. The misunderstanding is of a business rule, which is requirement knowledge
I4	Coding	Crash on names over 40 characters, so length handling is missing	Requirements (no maximum length given). Software must not crash on any input, so coding
I5	Coding	Age-60 rule is simple. The boundary was coded wrongly	Requirements (when someone “turns 60” is unstated). The rule is common knowledge, so coding
I6	Design	Two doctors overwrite shared data, which is a decision about shared state	Coding. Data sharing is an architectural choice
I7	Documentation	Manual names a menu that does not exist	None
I8	Testing	Wrong expected value in the test	None
I9	Environment (external change)	Provider changed the API key	Design (no error handling or alerting). The trigger is external, but the design let it cause harm

I10	Requirements	No performance target given, so 25 seconds could not be judged a defect	Design/coding (slow loading). Without a target, nobody could call it a defect

Table 14: Pattern of origins

Origin	Incidents	Count	Share
Requirements	I2, I3, I10	3	30%
Design	I1, I6	2	20%
Coding	I4, I5	2	20%
Testing	I8	1	10%
Documentation	I7	1	10%
Environment	I9	1	10%

Interpretation: 5 of 10 incidents (50%) come from requirements and design, matching the 52 of 100 defects introduced in those phases. Only 2 incidents are coding defects, yet coding is where the team put all its (self-check) effort. Three incidents are not code defects at all. I1 and I6 share one cause (shared data under simultaneous use), which suggests a design weakness that may exist elsewhere.

Where to invest: requirement reviews, design inspections and independent testing, not more coding skill. These are also the cheapest points to fix defects (₹500 and ₹1,500 against ₹40,000 in production).

TASK 8 – Evaluate Defect Classes and Severity (4 Marks)

Ans:

Table 15: Classification scheme

Class	Sub-classes	Incidents

Requirements/specification	Business rule, functional description, interface description, non-functional (performance)	I2, I3, I10
Design	Control logic, data and shared state, module interface, external interface	I1, I6, I9
Coding	Control logic (boundary), data/input handling, algorithm, initialisation	I4, I5
Testing	Test-case design, test oracle, test procedure, test harness	I8
Documentation (added)	User manual, help text	I7

Scales used. *Severity* is the harm if the defect occurs: Critical = patient safety/privacy risk; High = key function wrong or crashing, or money wrongly taken; Medium = function wrong but a workaround exists; Low = cosmetic. *Priority* is the order of fixing, based on severity plus how often it happens, workaround, fix cost and risk of delay (P1 first, P4 last).

Table 16: Class, severity and priority of every incident

Incident	Class / sub-class	Severity	Priority	Reason
I1	Design / concurrency	High	P2	Disrupts the queue, but needs the same second and doctor, and reception can issue a new token
I2	Requirements / missing business rule	Medium	P2	Patients travel to a closed OPD, but no safety risk. Cheap fix, many affected
I3	Requirements / wrong business rule	High	P1	Every booking overcharged, with financial, trust and possible tax/legal impact
I4	Coding / input handling	High	P3	App crashes, but only for long names, a shorter name works around it, and the fix is easy

I5	Coding / boundary	Medium	P3	Small group affected, but it is a fairness issue
I6	Design / shared state	Critical	P1	One patient's details shown on another's prescription is a safety and privacy breach
I7	Documentation	Low	P2	No data harm, but every patient reading the manual is misled. Fix takes minutes
I8	Testing / oracle	High	P2	False "Pass" hides a possible refund defect and makes other test results doubtful
I9	Design / external interface	Medium	P3	Missed reminders cause missed visits, no direct harm. Needs alerting and key handling
I10	Requirements / performance	Medium	P3	Slow but correct. Needs a target and profiling, planned next cycle

Why severity and priority differ: severity measures the damage if the defect occurs. Priority also depends on frequency, workaround, fix cost and what else is waiting. **I7** is Low severity but P2: it harms no data, but every reader is misled and the fix takes minutes. **I1** is High severity but only P2: the damage is serious, but it needs two bookings in the same second and reception can reissue a token, so I3 and I6 go first. Ranking by severity alone would give the wrong order.

Most harmful class, and where to start: the most harmful defects are the design/shared-data defect I6 (Critical) and the money-related requirement defect I3. The team's own practice did not reflect this: it had no severity scale, only visible errors in a demo, so a crash and a privacy leak would have been treated alike.

Recommendation: target the **requirements/specification class first**, because (1) it holds 30 of the 100 defects, (2) only 2 of those 30 were caught before release (6.7%), and (3) it is the cheapest to fix early (₹500 against ₹40,000). I6 should be hot-fixed immediately, and design inspection of shared data should follow.

TASK 9 – Design and Appraise a Defect Repository (3 Marks)

Ans:

Position: the repository is the project's memory. It must let the team answer what went wrong, where it began, why it was not caught earlier, how serious it was, how it was fixed and how to stop it recurring.

Table 17: Repository fields and why each is needed

Field group	**Fields**	**Why**
Identification	Defect ID, title, date, reporter, source (patient, tester, review, monitoring)	Unique reference. Shows who finds defects, so detection weaknesses are visible
Description	Steps to reproduce, expected result, actual result, attachments	Lets a developer reproduce the failure without questions. Separates failure from expected behaviour
Location	Module, release/build, environment, device/browser	Shows where defects cluster and which release holds them
Classification	Class and sub-class	Basis for analysis and checklists
Origin and detection	Phase introduced, phase detected, detection method	Gives the phase-containment and escape figures used in Task 3
Rating	Severity, priority, patient-safety/privacy flag, patients affected	Sets the order of work and highlights serious risks
Handling	Status, owner, dates	Tracks to closure and shows ageing
Cause and fix	Root cause (error), fix description, effort, cost	Enables cost-per-phase figures and learning from causes
Traceability	Requirement ID, test case ID, related or duplicate defects	Shows which requirement or test was missing and links repeat defects

Prevention	Preventive action, owner, status	Turns each defect into a process improvement

How it supports

- **Analysis:** Pareto charts by class, module and phase show where to focus.
- **Process improvement:** many requirement defects found late show reviews are weak. Many reopened defects show weak fixing. The data set targets, for example production escapes below 20%.
- **Prevention:** each cause becomes an action. I1 and I6 would add a design-checklist item "shared data under simultaneous use" and a concurrency test every release.
- **Test design:** past defects supply regression cases and ideas for similar modules.
- **Management:** trends replace "the demo went well" in release decisions.

Table 18: E-mail and spreadsheet versus repository

Aspect	**E-mail / spreadsheet**	**Repository**
Structure	Free text, different columns per person	Fixed fields and lists
Duplicates	Same problem reported repeatedly	Duplicates linked to one defect
Workflow	No status or owner, defects get lost	Status, owner, due dates
History	No change log, conflicting versions	Every change logged
Analysis	Manual counting by class, phase or cost	Built-in filters and reports
Traceability	No link to requirement, test or build	Linked to all three
Privacy	Patient details circulate in mail and sheets	Access control and masking

Judgement: it is worth the effort. A free or low-cost tool works, entering fields takes a few minutes per defect, and the hospital gets the data it needs to decide where to invest.

TASK 10 – Evaluate Defect Examples and Prevention Strategies (4 Marks)

Ans:

I chose one incident from each of three classes: **I3** (requirements), **I6** (design) and **I5** (coding).

Table 19: Root cause and prevention

Incident	**Root cause**	**Single process change that would have prevented it**
I3 – fee ₹500 shown, ₹550 charged	Fee and GST rule never documented or agreed with billing, and implemented twice from different readings	Requirement review with billing staff, a written fee-rule table, fee calculated in one place, table used as the test oracle
I6 – wrong patient on prescription	Shared data across users, with no design review for simultaneous use	Design inspection with checklist item "shared data and simultaneous users", plus a concurrency test on prescription save
I5 – no senior priority at 60	Age rule coded without a boundary check, no code review, no 59/60/61 test	Coding standard for date/age logic, peer code review, boundary-value unit tests

Cost-benefit. Assumptions: ₹800 per person-hour. For each extra defect caught in its own phase, saving = ₹40,000 (production) minus that phase's cost. This is an upper bound, because not every missed defect would reach production. Extra defects caught come from the review model in Task 3 (50% review efficiency), and for coding from raising detection from 17.6% to 35% of the 56.5 defects present.

Table 20: Value of each strategy (per release)

Strategy	Effort (hrs)	Cost (₹)	Extra defects caught	Saving each (₹)	Total saving (₹)	Benefit ÷ cost
Requirement review	50	40,000	13 (15 instead of 2)	39,500	5,13,500	12.8
Design inspection	75	60,000	15.5 (18.5 instead of 3)	38,500	5,96,750	9.9
Coding standard + review + boundary tests	75	60,000	≈ 9.8	37,000	≈ 3,62,600	6.0

Savings overlap, since a defect can only be caught once, so they must not be added together. All three return at least 6 times their cost.

Best value: requirement review (ratio about 12.8). It needs only a few hours with billing and OPD staff, needs no tools, targets the largest group (30 of 100 defects) and would have prevented I2, I3 and I10. It also feeds the others, because test oracles come from reviewed requirements. I would still keep design inspection (ratio about 9.9), because I6 is a Critical privacy defect whose true cost is far above the ₹40,000 used here.

TASK 11 – Recommend and Defend a Quality Improvement Plan (4 Marks)

Ans:

Basis from Tasks 1–10: 52% of defects start in requirements and design, only 9.6% of these are caught early, production defects take 91.9% of cost, and there is no tester, exit criterion or repository.

Table 21: Plan in priority order

Priority	Change	Role of tester / reviews / repository	Addresses	Effort

1	Release gate with written exit criteria (no open critical/high defect, high-risk tests done, ≥95% pass), signed by the hospital	Tester gathers evidence, management decides	The demo-based approval, all incidents	A few hours per release
2	Rule review with billing and OPD staff (fees, working days, age rules, performance target) and a business-rule table	Tester and developers review. The table serves as the oracle	I2, I3, I5, I10	≈ 50 person-hours
3	Appoint an independent tester	Test plan, boundary and equivalence tests, system tests, review of test cases (I8)	I4, I5, I7, I8, I10	One person
4	Design inspection of shared data, concurrency and external interfaces, with a concurrency test and a load test	Tester and two developers, checklist kept in the repository	I1, I6, I9	≈ 75 person-hours
5	Defect repository (Task 9 fields) and monthly review by class, phase and cost	Whole team, tester owns reports	Learning from every incident	1 day set-up, 5 min per defect
6	Coding standard, peer review and boundary unit tests	Developers	I4, I5	≈ 75 person-hours
7	Manual walkthrough on the live app, and SMS interface monitoring	Tester, operations	I7, I9	A few hours

Expected impact. Model: requirements and design reviews remove 50% of defects present, coding detection rises from 17.6% to 35%, testing detection rises from 26.7% to 60%.

Phase	Present	Detected	Cost (₹)
Requirements	30	15.0	7,500
Design	37	18.5	27,750
Coding	56.5	19.8	59,325
Testing	41.7	25.0	2,00,280
Production	21.7	21.7	8,67,600
Total			**≈ 11,62,455**

Table 22: Outcome per release

Measure	Last release	With plan
Defects found before release	40	≈ 78
Defects reaching patients	60	≈ 22
Cost of fixing defects (₹)	26,10,500	11,62,455
Cost of the plan (tester ₹2,40,000 for 6 months at ₹40,000/month; reviews ₹1,00,000; code review and unit tests ₹60,000; repository/tools ₹20,000)	–	4,20,000
Total cost (₹)	26,10,500	15,82,455
Net saving (₹)		**10,28,045 (about 39%)**

Defects reaching patients drop by about 64%. This excludes unquantified gains such as fewer safety and privacy incidents.

Trade-offs and risks

- Reviews and the gate take calendar time, and the hospital must accept a gate that can delay a release.
- The 50%, 35% and 60% efficiencies are assumptions. The repository will give real figures to update them.
- Reviews can become a tick-box exercise, so checklists and recorded findings are needed.
- Tester and developers must work as one team, with defects reported without blame.
- Costs of patient harm are not counted, so the real benefit is higher.
- The model rests on data from a single release.

Workable for five developers? Yes. The developer effort is about 125 hours (reviews) plus 75 hours (code review and unit tests) = 200 hours. Five developers over 6 months give $5 \times 6 \times 20 \times 8 = 4{,}800$ hours, so the plan uses about 4% of developer time. Reviews are short, checklist-driven and focused on high-risk parts, and the repository tool can be free. The only large cost is the tester, who is also the main source of savings.

TASK 12 – Re-evaluate and Adapt the Quality Process (8 Marks)

A. Impact Analysis (2 Marks)

Ans:

Table 23: What has changed

Aspect	**Original**	**Changed**
Release rhythm	Every 6 months	Every 2 weeks (26 a year, about 13 times as often)

Interfaces	Own app and SMS provider	Plus the national health ID system run by an external agency
Data	One hospital's appointments and prescriptions	Health records shared across hospitals
Control	Team controls all code and releases	Agency controls when and how the ID system changes

Assumptions in my plan that are affected

- *Whole-document reviews (≈125 hours) each release:* cannot be repeated every two weeks. Reviews must be small and tied to each change.
- *Manual testing by one tester:* a manual regression run of 400 cases × 6 minutes is 40 hours, half of a tester's 80-hour cycle.
- *A test bed the team controls:* the agency's sandbox may behave differently from the live system.
- *A repository for one system:* some defects will now lie in the agency's system or the contract between the two.
- *₹40,000 per production defect:* too low when a bad record can spread to other hospitals and every release is live to 6,000 patients a day.

Why frequent releases change the tester's role: there is no "testing phase at the end" any more. Testing runs alongside development from story refinement onwards. Repetitive checks are automated and run on every change, and the tester spends time on risk analysis, exploratory testing, automation and the release decision. Releases must be reversible and monitored.

Why shared records raise severity: an I6-type defect (another patient's data in a document) becomes a confidentiality breach across hospitals, with legal duties and possible harm to a patient treated on wrong information. Design and data defects (shared state, identity matching), interface defects, access-control defects (previously Medium) and consent/identity requirement defects all become Critical. Defects like I7 stay Low.

B. Evaluation of the Original Plan (2 Marks)

Ans:

Table 24: Fit of the plan to a two-week cycle (tester capacity 80 hours per cycle; hour figures are my estimates)

Planned activity	**Fits?**	**Change needed**
Review of all rules (≈50 hrs)	No	Review only the rules in that cycle's stories (30–45 min each), and keep the rule table current
Design inspection (≈75 hrs)	Partly	Do it only for stories touching shared data, identity or external interfaces
Manual testing by one tester	No	Per-cycle need: story review 10 + test design/execution for changes 35 + manual regression 40 + ID-system and privacy checks 20 + reporting/gate 5 = **110 hrs against 80 available, about 38% over capacity**
Release gate	Yes	Keep it, and automate the evidence
Code review and unit tests	Yes	Keep as part of "definition of done"
Defect repository	Partly	Needs new fields (below)

New defect origins from the external health ID system

- *Contract/version defects:* the agency changes a field, version or rule without notice (same pattern as I9).
- *Data-mapping and identity defects:* wrong patient matching, inconsistent name/date formats, duplicate IDs.
- *Availability/speed:* the service is slow or down, so booking freezes or is lost.
- *Authentication and consent defects:* expired tokens, missing consent, wrong access rights.

- *Environment differences:* the agency's test environment differs from live.
- *Privacy/security defects:* records exposed in transit or in logs.

Is the repository adequate? No. It assumes every defect is in the team's own code. It needs these extra fields: internal / agency / interface owner, interface version or contract reference, agency ticket number and response date, data sensitivity and number of patients/hospitals affected, whether it must be reported to an authority, pipeline stage where it was found, and time to detect and restore service.

C. Adaptation or Redesign (2 Marks)

Ans:

Decision: MODIFY.

- I do not **retain** the plan, because its reviews and manual testing are about 38% over capacity in a two-week cycle and it has no way to handle an external system.
- I do not **redesign** it, because the evidence behind it still holds: over half the defects start in requirements and design, early detection is far cheaper (₹500 / ₹1,500 against ₹40,000), and a tester, release gate and repository are still needed.
- What changes is how small each activity is and what gets automated.

Revised two-week cycle (Figure 2, draw as a loop):
Story refinement with rule check → design check for risky stories → code + unit tests + peer review → automated pipeline (regression and contract tests against the health ID sandbox) → tester's exploratory, load/concurrency and privacy checks → release gate → small-group release with monitoring and rollback → defect repository and retrospective → feedback into the next cycle.

- *Story refinement:* each story gets acceptance criteria and a rule check by the tester and the hospital's rule owner (30–45 minutes).
- *Design check:* only for shared data, identity or external interfaces, using the checklist.

- *Pipeline:* critical paths (booking, token, fee, prescription, reminder) are regression-tested on every change. The 40-hour manual regression becomes an automated run of a few minutes, leaving about 5 hours of upkeep. The per-cycle need then drops to about 75 hours, within the 80 available.
- *Release and rollout:* exit criteria come from pipeline results. Release goes to a small group first and can be rolled back. SMS and health ID interfaces are monitored with alerts.

Tester's revised role: an embedded test engineer and quality coach. They analyse risk per story, build and maintain automated regression and contract tests, do exploratory and privacy testing, own the release-gate evidence and repository reports, and are the contact point with the agency for interface problems.

Principles that gain importance

- *Early testing and planning:* two weeks leave no time for late correction.
- *Repeatable, automated regression:* every release must re-prove existing behaviour.
- *Testing depends on context:* health data and money need stricter testing.
- *Defect clustering:* interfaces and shared data are where new defects will gather.
- *Test invalid inputs and failures at interfaces:* slow, wrong or missing agency responses are normal cases to test.

Table 25: Classifying and tracking external-system defects

Item	How handled
Class	New "External interface" class with sub-classes: contract/version, data mapping, availability/performance, authentication/consent, environment difference
Owner	Internal (our adapter), agency (their service) or joint (contract unclear). This decides who fixes it
Severity	By patient and data impact. Any wrong record or privacy exposure is Critical

Tracking	Our defect ID linked to the agency's ticket. Response times logged. Workarounds recorded (queue, retry, manual process)
Measures	Time to detect, time to restore, external defects per release, share caused by agency changes
Prevention	Contract tests on every build, production monitoring, a named agency contact and a shared release calendar

D. Evidence-Based Defence (2 Marks)

Ans:

Table 26: Evidence behind the decision to modify

Evidence	**What it shows**	**Effect on decision**
Defect data and cost	Patients found 60% of defects, which was 91.9% of cost (₹24,00,000 of ₹26,10,500). A production defect costs 80 times a requirements defect	Early-detection logic stays, so the plan is not discarded
Origins and classes	50% of incidents and 52% of defects come from requirements and design. Coding is only 20% of incidents	Keep requirement and design checks, but small and per-story
Release frequency	26 releases a year. Manual per-cycle need of 110 hours against 80 available	An unchanged plan would fail, so automation and smaller reviews are needed
Patient safety and privacy	Shared records turn I6-type defects into cross-hospital breaches (Critical)	Add design checks, privacy testing and a strict release gate
External agency	New defect origin outside the team's control	Add contract tests, monitoring and the new repository class

Team size and capacity	Five developers (about 4,800 hours in 6 months) plus one tester. The plan uses about 4% of developer time	A full redesign with a larger test team is unaffordable

Reasoning: if I kept the plan unchanged, the team would either drop activities and return to the old situation (60 defects escaping) or delay every release. If I redesigned everything, I would throw away parts the data strongly support, at a cost five developers cannot carry. The modified plan keeps what is proven, shrinks it to fit two weeks and automates the repeated checks.

Expected benefit: the Task 11 estimate (about 22 defects reaching patients instead of 60, saving about ₹10 lakh per release-sized body of work) applies to the same quality activities at the new pace. The release gate, monitoring and rollback also limit any escaped defect to two weeks of exposure instead of six months.

Risks I accept, and how I monitor them

- The automated suite takes time to build, so early cycles have lower coverage. The tester starts with the critical paths.
- Review and test efficiencies are estimates. The repository will give real figures after about three cycles.
- The agency's behaviour is outside the team's control. Contract tests and monitoring give early warning.

Review trigger: if the repository shows that agency-interface defects dominate and the team cannot control them, I would reconsider and redesign the process around a dedicated integration-test group and a formal service agreement with the agency.